

Hybrid Movie Recommendation System

This report explains the implementation of a Hybrid Recommendation System using Content-Based Filtering, Collaborative Filtering, and a Streamlit User Interface.

1. Introduction

Recommendation systems are one of the most important machine learning applications used in platforms such as Netflix, YouTube, and Amazon. This project combines content-based filtering and collaborative filtering techniques to improve recommendation quality and personalization.

2. Dataset Description

The MovieLens dataset contains user ratings, movie titles, genres, and user interactions. The dataset helps the recommendation system learn user preferences and movie similarities.

3. Data Preprocessing

The preprocessing phase ensures clean and consistent data before training the recommendation models.

Data Loading Code

```
import pandas as pd

movies = pd.read_csv("movies.csv")
ratings = pd.read_csv("ratings.csv")
tags = pd.read_csv("tags.csv")

movies.head()
```

The code above loads the MovieLens dataset files using Pandas. Movies, ratings, and tags are imported separately for preprocessing and analysis.

4. Content-Based Filtering

Content-based filtering recommends movies similar to the movies that the user already likes. TF-IDF vectorization is used to convert genres and movie metadata into numerical vectors.

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

tfidf = TfidfVectorizer(stop_words='english')

movies['genres'] = movies['genres'].fillna('')
```

```
tfidf_matrix = tfidf.fit_transform(movies['genres'])

cosine_sim = cosine_similarity(tfidf_matrix, tfidf_matrix)
```

TF-IDF converts movie genres into feature vectors. Cosine similarity calculates how similar two movies are based on their content features.

5. Collaborative Filtering

Collaborative filtering predicts user preferences based on user-item interactions. Matrix factorization techniques such as SVD are used to discover latent features.

```
from surprise import SVD from surprise import Dataset
from surprise.model_selection import train_test_split

data = Dataset.load_builtin('ml-100k')

trainset, testset = train_test_split(data, test_size=0.2)

model = SVD()
model.fit(trainset)
predictions = model.test(testset)
```

The SVD algorithm learns hidden patterns between users and movies. The model predicts ratings for movies that users have not watched yet.

6. Hybrid Recommendation Engine

The hybrid model combines the outputs of content-based filtering and collaborative filtering using weighted averaging. This improves recommendation quality and balances personalization.

```
hybrid_score = (0.4 * content_score) + (0.6 * collaborative_score)
```

The collaborative filtering model was given a higher weight because it produced better prediction accuracy during testing.

7. Evaluation Metrics

Metric	Purpose
RMSE	Measures prediction error
MAE	Measures average absolute error
Precision	Measures recommendation relevance
Recall	Measures how many relevant movies were retrieved
F1-Score	Balances Precision and Recall

8. Streamlit User Interface

The Streamlit application allows users to interact with the recommendation system through a simple and modern interface.

```
import streamlit as st
```

```
st.title("Movie Recommendation System")

user_id = st.number_input("Enter User ID")

if st.button("Recommend"):
    st.write(recommend_movies(user_id))
```

Users can enter their User ID and instantly receive personalized movie recommendations.

9. Challenges Faced

- 1 Handling sparse user-item matrices
- 2 Combining two recommendation approaches efficiently
- 3 Improving recommendation accuracy
- 4 Reducing cold-start problems

10. Conclusion

The project successfully demonstrates the implementation of a Hybrid Movie Recommendation System using machine learning techniques. The combination of TF-IDF, cosine similarity, and SVD significantly improved recommendation performance and personalization quality.